

Web App Vulnerability Assessment

Task 1 — OWASP ZAP Scan Report

Target	OWASP Juice Shop (local instance)
URL	http://localhost:3000
Tool	OWASP ZAP by Checkmarx, v2.17.0
Date	11 August 2026
Prepared for	Indravex Technologies — Cyber Security Internship, Task 1
Prepared by	Prajwal

Summary

I ran ZAP's Spider + Active Scan against a local Juice Shop instance. It came back with 14 alert groups, but 4 of those were noise — background telemetry calls Firefox itself makes to Mozilla's servers during the scan, not anything to do with the app. Those are called out at the end so nothing gets double-counted.

That leaves 10 real findings. The one that matters most is a SQL injection in the product search endpoint — confirmed, not just suspected, because the server threw back a raw database error I could read.

What needs fixing first

SQL Injection — High risk / Low confidence

The search box (q parameter on /rest/products/search) doesn't sanitize input. Sending ' as the query made the server crash with a raw SQLite error, which exposed the exact query being built behind the scenes — a clear sign the input is going straight into the SQL string.

SQLITE_ERROR. Something about a syntax error near a parenthesis.

The issue is that the full query structure is visible, in the response.

To fix this we need to do a things.

- *We should use queries all the time.*
- *We have to check what the user is sending to the server to make sure it is okay.*
- *We should never show the real database errors to the user.*

Instead we should log these errors on our server and give the user a simple message that says something went wrong.

Content-Security-Policy Header Missing — Medium risk / High confidence

Content-Security-Policy Header Missing. Risk / High confidence

No CSP header is present anywhere, on the site. CSP is one of the ways to protect against XSS attacks because it tells the browser which sources are allowed to run scripts or load content.

Fix: add a Content-Security-Policy header that limits scripts and resources to trusted origins.

Session ID Exposed in URL — Medium risk / High confidence

A session ID is being sent in the URL of just in a cookie. Session IDs that are in the URL can be exposed through browser history, server logs and the Referer header.

Fix: make sure session IDs are stored in a Secure cookie. Remove the version that's, in the URL.

CORS Misconfiguration — Medium risk / Medium confidence

The API sends Access-Control-Allow-Origin: * — a wildcard that lets any website read its responses. Fine for public data, risky the moment an endpoint returns anything user-specific.

Fix: replace the wildcard with an explicit allow-list of trusted origins, or drop the header on endpoints that don't need it.

Missing Anti-Clickjacking Header — Medium risk / Medium confidence

No X-Frame-Options or CSP frame-ancestors directive is set, so pages can be embedded in an iframe on another site — the basis of clickjacking attacks.

Fix: set X-Frame-Options: SAMEORIGIN (or DENY) or the equivalent CSP directive.

Lower-priority findings

The rest of the alerts are low risk or purely informational — worth knowing about, but none need urgent action:

- Private IP disclosure — internal addresses (192.168.99.100) showed up in a response body. Strip hard-coded internal IPs from client-facing responses.
- Unix timestamp disclosure — a raw system timestamp appeared in a response. Low sensitivity here, but avoid exposing raw timestamps where not needed.
- Modern web application (info) — ZAP correctly flagged this as a JS-driven SPA, which is why some tools like Nikto return unreliable results against it (SPAs tend to return 200 instead of 404 for unknown routes).
- User-Agent fuzzing (info) — server was probed with spoofed User-Agent strings; no behavioral differences of concern turned up.
- Suspicious comment (info) — a developer comment containing the word "query" was found in the JS bundle. Worth stripping debug comments from production builds as routine hygiene.

Excluded from scope

Four alerts — missing HSTS header, missing X-Content-Type-Options, "retrieved from cache," and cache-control directive warnings — were all triggered by Firefox's own telemetry calls to Mozilla domains, not by Juice Shop. ZAP doesn't filter these automatically, so I checked each alert's target before excluding it, rather than reporting alert titles at face value.
